

Kleidungslager – Screenshot-Dokumentation

Projekt: Kleidungslager

Autor: Ibrahim Danisman · Matrikelnummer 578949

Dokumentation der zentralen Use-Cases mit sichtbarem Umsetzungsnachweis.

1. Anmelden

Der Benutzer meldet sich über E-Mail und Passwort an. Geschützte Seiten wie Dashboard und Einstellungen werden erst danach freigeschaltet.

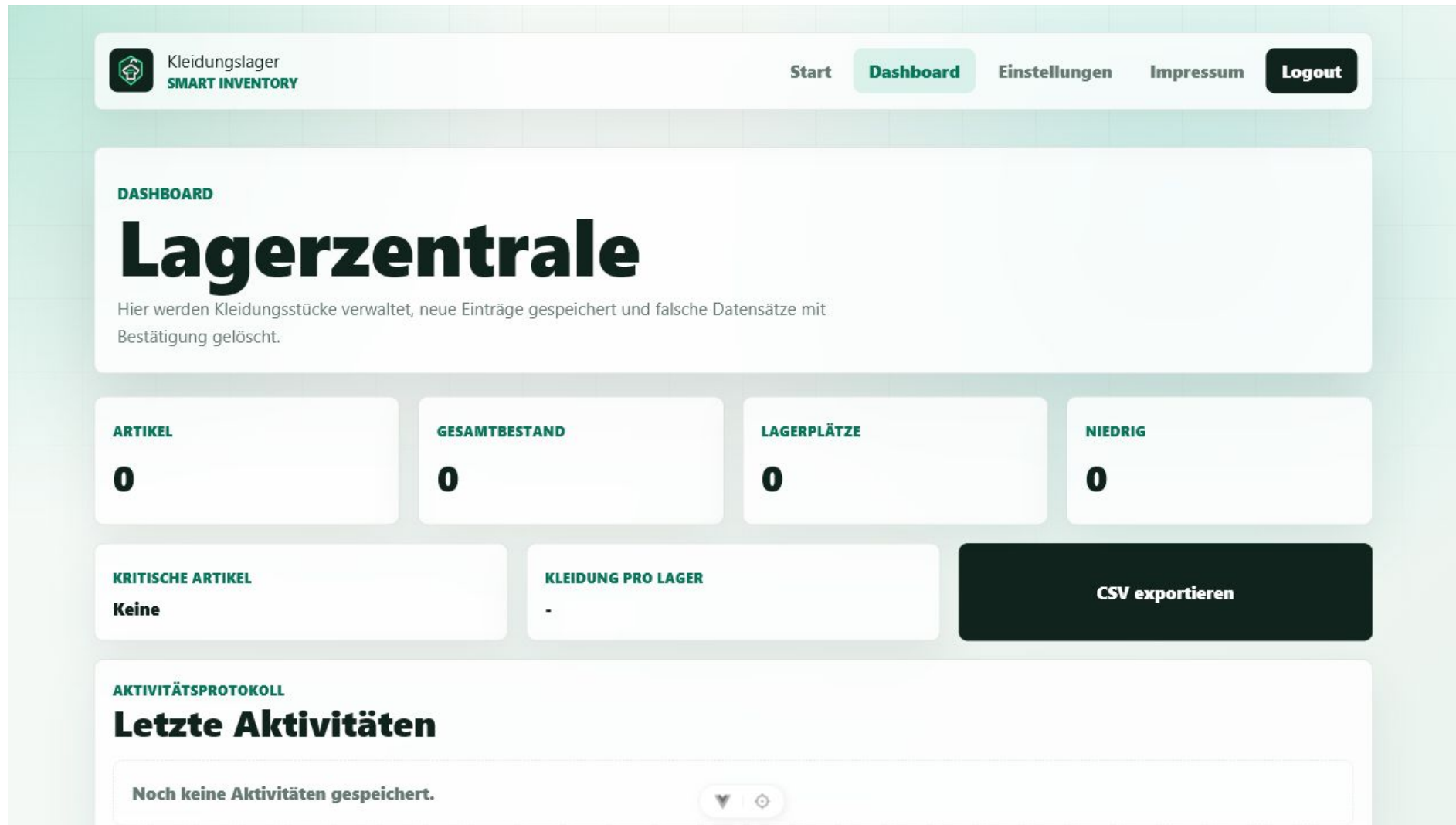
Umsetzung: Frontend: src/views/LoginView.vue, src/auth.ts und src/router/index.ts

The screenshot shows a web application interface for 'Kleidungslager SMART INVENTORY'. The header bar is light green and contains the logo on the left and navigation links 'Start', 'Impressum', and 'Login' on the right. The 'Login' link is highlighted in a darker green. The main content area has a light green background with a subtle grid pattern. In the center, there is a white login card with a dark green border. The card is titled 'SICHERER BEREICH' in small green letters, followed by 'Login' in large, bold black letters. Below the title, a subtitle reads: 'Demo-Anmeldung für dein Dashboard. Nach dem Einloggen wird die Lagerverwaltung freigeschaltet.' The card contains two input fields: 'E-Mail' and 'Passwort', both with rounded rectangular borders. Below these fields is a large, dark green button with the text 'Einloggen' in white. At the bottom of the card, a line of text provides demo credentials: 'Demo: Ibrahim.Danisman@Student.HTW-Berlin.de · Passwort: demo'. The overall design is clean and modern, using a green color palette.

2. Dashboard und Lagerkennzahlen

Das Dashboard zeigt Artikelanzahl, Gesamtbestand, Lagerplätze, niedrige Bestände, kritische Artikel und das Aktivitätsprotokoll.

Umsetzung: Frontend: src/views/DashboardView.vue und src/components/KleidungsListe.vue



3. Kleidungsstück erstellen und Eingaben validieren

Über die Eingabemaske wird ein neues Kleidungsstück gespeichert. Ungültige oder leere Pflichtfelder werden direkt am Feld markiert. Das Backend prüft dieselben Regeln erneut.

Umsetzung: Frontend: KleidungsListe.vue und formularValidierung.ts; Backend: Kleidungsstuecke.java, ApiExceptionHandler.java und KleidungController.java

NOCH KEINE AKTIVITÄTEN GESPEICHERT.

 Kleidungslager
SMART INVENTORY

Start Dashboard Einstellungen Impressum Logout

NEUER EINTRAG

Kleidungsstück speichern

Bitte korrigiere die markierten Eingaben.

Artikelnummer / Barcode

z.B. KL-0001 oder Barcode

Scannen

Bezeichnung

Bitte gib eine Bezeichnung ein.

Größe

M

Kategorie

HEMD

Farbe

LIVE-BESTAND

Kleidungsstücke im Lager

API Barcode suchen

Suche Kategorie Größe Lager

Suche nach Kle Alle Alle Alle

Sortierung Ansicht

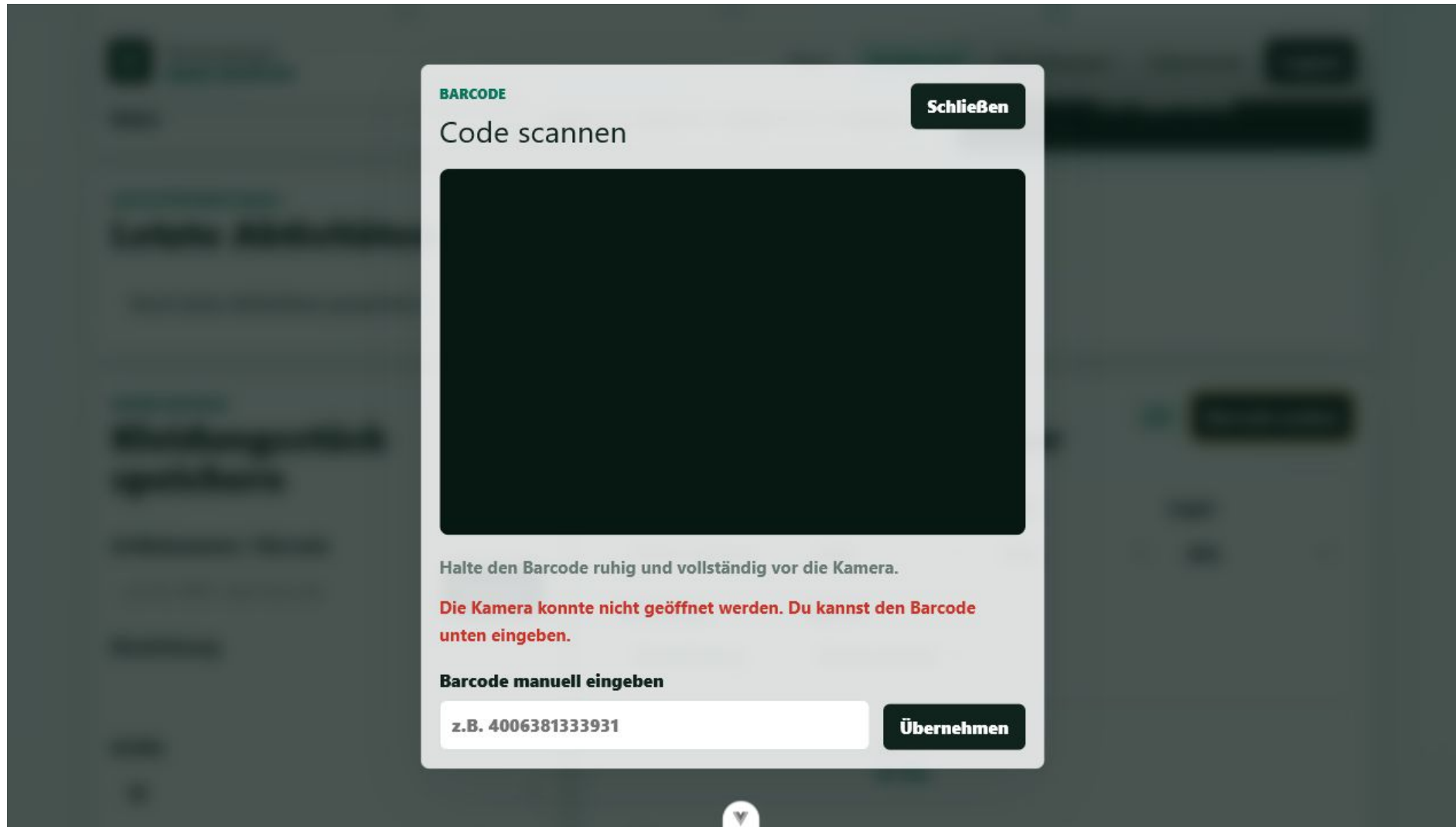
Bezeichnung Kartenansicht

Keine passenden Kleidungsstücke gefunden.

4. Barcode scannen und Produkt finden

Ein Barcode kann über die Kamera oder als manuelle Eingabe übernommen werden. Bei der Produktsuche wird eine exakte Artikelnummer gesucht und bei einem Treffer die Detailseite geöffnet.

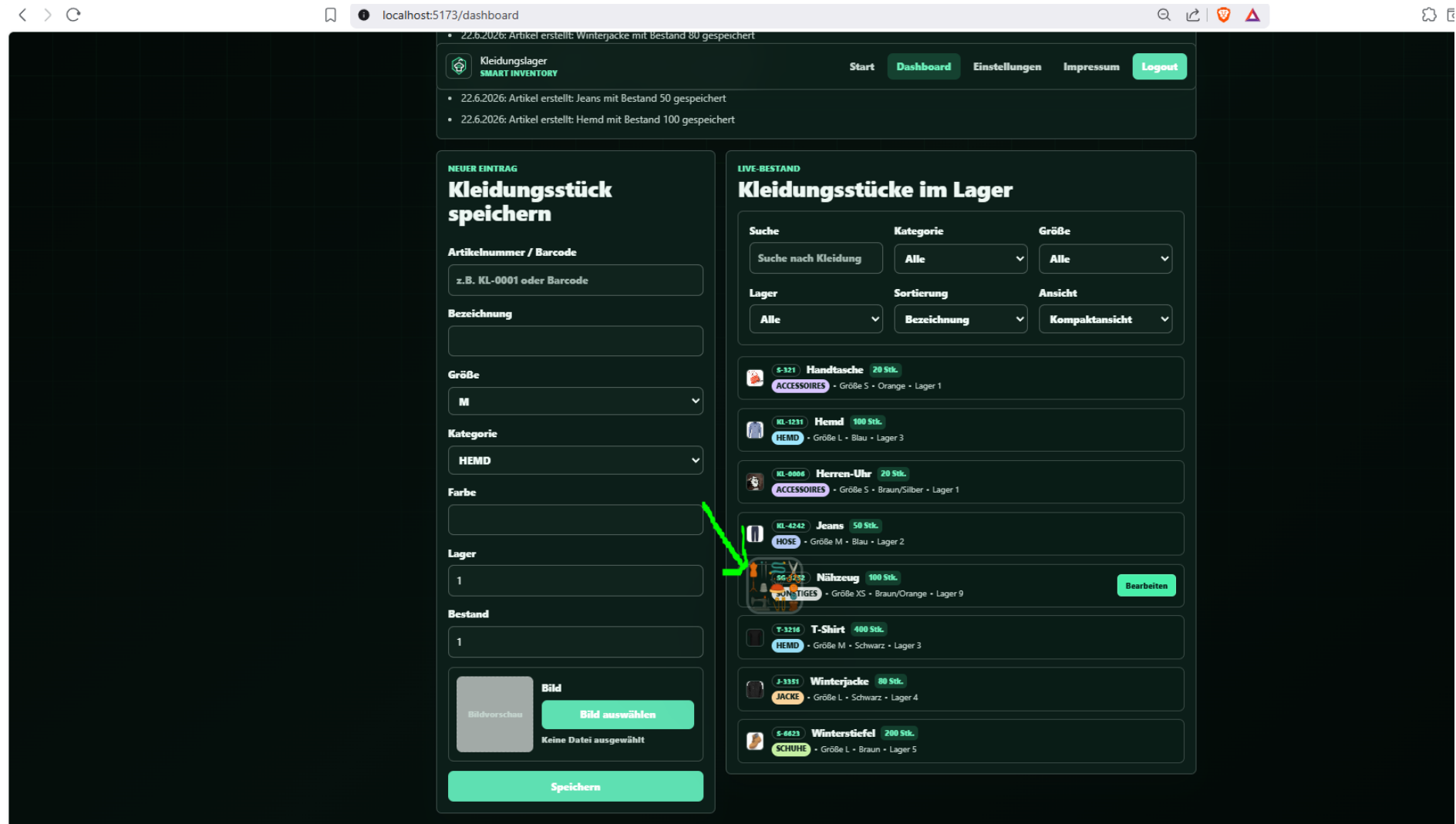
Umsetzung: Frontend: BarcodeScanner.vue und barcode.ts; Backend: GET /api/kleidung/artikelnummer/{artikelnummer}



5. Live-Bestand und Kompaktansicht

Gespeicherte Kleidungsstücke werden aus der REST-Schnittstelle geladen. Die Kompaktansicht stellt viele Artikel übersichtlich dar und richtet die Bestandszahlen einheitlich aus.

Umsetzung: Frontend: KleidungsListe.vue; Backend: GET /api/kleidung



6. Details anzeigen, bearbeiten und löschen

Auf der Detailseite können Stammdaten, Bestand, Lager und Bild bearbeitet werden. Der Löschen-Button entfernt den Artikel erst nach einer Bestätigung.

Umsetzung: Frontend: KleidungsDetailView.vue; Backend: PUT /api/kleidung/{id}/bestand und DELETE /api/kleidung/{id}

The screenshot displays a web application interface for managing clothing items. The browser address bar shows 'localhost:5173/kleidung/3'. The application header includes a logo 'KleidungsLager SMART INVENTORY' and navigation links: 'Start', 'Dashboard', 'Einstellungen', 'Impressum', and a 'Logout' button.

A 'Zurück' button is located at the top left of the content area. The main content area features a large image of an orange handbag on the left. To its right, the item details are displayed:

- Item ID:** S-321
- Title:** Handtasche
- Category:** ACCESSOIRES
- Size:** Größe S
- Color:** Orange
- Warehouse:** Lager 1
- Stock:** 20 Stk.
- Article Number / Barcode:** S-321
- Designation:** Handtasche
- Size (dropdown):** S
- Category (dropdown):** ACCESSOIRES
- Color:** Orange
- Stock:** 20
- Warehouse:** 1

At the bottom of the details section, there are four buttons: 'Aktualisieren' (green), 'Abbrechen' (grey), 'Bild ändern' (green), and 'Bild entfernen' (grey). A red 'Löschen' button is located below the 'Aktualisieren' button.

Below the details section, there is an 'AKTIVITÄTSPROTOKOLL' section titled 'Dieser Artikel'. It contains a message: 'Hier werden Änderungen zu diesem Kleidungsstück gesammelt, zum Beispiel wenn Bestand, Lager oder Bild geändert werden.' and a list of activities:

- 22.6.2026: Artikel erstellt: Handtasche mit Bestand 20 gespeichert

7. Einstellungen verwalten

Profilwerte, Grenze für niedrigen Bestand, Standardsortierung, Kompaktansicht und Dunkelmodus können angepasst werden.

Umsetzung: Frontend: src/views/SettingsView.vue und src/settings.ts

 Kleidungslager
SMART INVENTORY

StartDashboardEinstellungenImpressumLogout

EINSTELLUNGEN

Kontrolle über dein Lager

Hier passt du dein Demo-Konto und das Verhalten des Dashboards an.

KONTO

Profil bearbeiten

Name

Ibrahim Danisman

E-Mail

Ibrahim.Danisman@Student.HTW-Berlin.

Speichern

DASHBOARD

Lagerregeln

Niedriger Bestand ab

5

Standardsortierung

Bezeichnung

☒ API-Hinweis im Dashboard anzeigen

☐ Kompaktere Kleidungskarten verwenden

☐ Dunkelmodus verwenden

SYSTEM

Weitere Optionen

- Bestandswarnung wird direkt im Dashboard angewendet.
- Profilwerte bleiben im Browser gespeichert.
- Die Einstellungen sind eine Frontend-Demo ohne Backend-Login.

Zurücksetzen

8. Impressum anzeigen

Das Impressum stellt Name, Matrikelnummer, Hochschul-E-Mail-Adresse und Hochschule dar.

Umsetzung: Frontend: src/views/ImpressumView.vue

The screenshot displays the 'Impressum' (Legal Notice) page of the 'Kleidungslager SMART INVENTORY' application. The page is part of a web interface with a light green background and a white content area. At the top, a navigation bar contains the application logo and name, and several menu items: 'Start', 'Dashboard', 'Einstellungen', 'Impressum' (which is highlighted in green), and 'Logout' (in a black button). Below the navigation bar, the page is titled 'RECHTLICHES' in green, followed by 'Impressum' in large black font. The main content area consists of four stacked white boxes, each containing a label and a value: 'Name' with 'Ibrahim Danisman', 'Matrikelnummer' with '578949', 'E-Mail' with 'Ibrahim.Danisman@Student.HTW-Berlin.de', and 'Hochschule' with 'HTW-Berlin'. At the bottom center of the page, there is a small white button with a downward arrow and a circular icon.

9. Suchen, filtern, sortieren und Ansicht wechseln

Der Live-Bestand kann nach mehreren Begriffen durchsucht, nach Kategorie, Größe und Lager gefiltert sowie nach Bezeichnung, Bestand, Lager, Kategorie oder Größe sortiert werden.

Umsetzung: Frontend: src/components/KleidungsListe.vue

Kleidungsstuck speichern

Artikelnummer / Barcode
z.B. KL-0001 oder Barcode

Bezeichnung

Größe
M

Kategorie
HEMD

Farbe

Lager
1

Bestand
1

Bild
Bildvorschau
Keine Datei ausgewählt

LIVE-BESTAND

Kleidungsstücke im Lager

Suche: Suche nach Kleidung
Kategorie: Alle
Größe: Alle
Lager: Alle
Sortierung: Bezeichnung
Ansicht: Tabellenansicht

ARTIKELNUMMER	BEZEICHNUNG	Bestand	Lager	FARBE	LAGER	ID
S-321	Handtasche		Orange	Lager 1	2	
KL-1231	Hemd	HEMD	L	Blau	Lager 3	1
KL-0006	Herren-Uhr	ACCESSOIRES	S	Braun/Silber	Lager 1	2
KL-4242	Jeans	HOSE	M	Blau	Lager 2	5
SG-3252	Nähzeug	SONSTIGES	XS	Braun/Orange	Lager 9	1
T-3216	T-Shirt	HEMD	M	Schwarz	Lager 3	4
J-3351	Winterjacke	JACKE	L	Schwarz	Lager 4	8
S-6623	Winterstiefel	SCHUHE	L	Braun	Lager 5	2

10. Automatisierte Tests

Frontend- und Backend-Tests laufen lokal sowie bei Push und Pull Request automatisch über GitHub Actions.

Frontend: `.github/workflows/frontend-tests.yml`

Backend: `.github/workflows/backend-tests.yml`

Frontend: 10 Vitest-Tests, TypeScript-Prüfung und Produktions-Build

Integration: GET-Ladevorgang, Formularvalidierung, POST-Erstellung und Barcode-Navigation

Backend: 7 JUnit-Tests mit Validierung, Controller-Endpunkten und Barcode-Suche

Integration: Spring Boot mit echtem HTTP-Port und temporärer PostgreSQL-16-Datenbank über Testcontainers

CRUD: POST erstellen, GET abrufen, Barcode suchen, PUT aktualisieren und DELETE löschen

Die Render-Datenbank wird von den Tests nicht verwendet oder verändert.